LLM-Assisted Reranking in Recommender Pipelines: An Empirical Study of Dependability, Fallback Mechanisms, and Prompt Sensitivity at Inference Time

Vladimir Kalmykov

Abstract

This report presents an empirical study of LLM-based listwise reranking in a sequential recommendation pipeline with a focus on dependability implications and tradeoffs rather than quality improvement. A fixed SASRec baseline trained on MovieLens-1M generates top-20 candidate lists, which three open-weight LLMs - Qwen2-7B-Instruct, Nemotron-Mini-4B-Instruct and gpt-oss-20b - attempt to rerank into top-10 recommendations at inference time without fine-tuning. The study compares four configurations: baseline SASRec recommendation, naive LLM deployment, hard fallback on any detected parse failure and a tiered fallback that distinguishes recoverable from fatal errors. The tiered mechanism recovers measurable quality that hard fallback unnecessarily discards by correcting LLM outputs with fixable errors like duplicates.

The original hypothesis - that self-consistency across repeated LLM calls could serve as a reliability and quality signal - failed to find support: no significant correlation between output consistency and quality was observed in the configurations tested. However, the research yielded several other findings: prompt wording is approximately 100x more destabilising than candidate input ordering, contradicting emphasis on position bias in literature. Aggressive diversity reduces popularity bias, but paradoxically decreases genre diversity, revealing that LLM conflates diversity and obscurity. A model-capability-dependability tradeoff emerges across the three models: it is instruction-tuning quality, not parameter count, that determines format compliance. The 7B instruct-tuned Qwen achieves 100% valid, though conservative output at low temperature; the 4B Nemotron 45%, and the 20B base model 0% due to interface incompatibility. The report discusses implications for future fallback and anomaly detection in LLM-augmented recommender pipelines.

Due to irregular gitlab commits, a timeline is added in the appendix.

1 Introduction

Recommender systems increasingly adopt multi-stage pipeline architectures [18], with a recent trend integrating LLMs for reranking [3], zero-shot recommendation [13], and bias mitigation [8]. However, LLMs introduce failure modes invisible to conventional monitoring: syntactically unparseable output, hallucinated item identifiers not in the candidate set, non-deterministic behaviour across identical inputs, or output that is technically valid but no better than the baseline - a pattern analogous to *gray failure* in distributed systems [2], *where a component appears operational but delivers degraded service.* These risks are rarely studied. The LLM reranking literature overwhelmingly assumes the model produces usable output and evaluates quality in aggregate, without examining per-query reliability or proposing mechanisms for when it does not.

This project began with a specific hypothesis: that *self-consistency* - the agreement among multiple LLM outputs for the same input - could serve as a signal to decide whether to trust the LLM's reranking or fall back to

the base ranker. This hypothesis, borrowed from reasoning tasks where self-consistency successfully detects unreliable outputs, failed in the reranking setting. The reasons it failed, and what the investigation revealed instead, form the core contributions of this report.

Specifically: *(1) a two-tier failure taxonomy with tiered fallback that recovers quality lost by hard fallback; (2) a documented negative result on self-consistency as a reliability signal; (3) a perturbation hierarchy showing prompt wording dominates candidate ordering by 100×; and (4) evidence that aggressive diversity prompting conflates diversity with obscurity and produces illusory fairness gains through degradation of the advantaged group.*

These findings emerged from four research questions:

RQ1: Does LLM-based reranking improve recommendation quality?
RQ2: How stable is LLM reranking under repeated inference, temperature variation, prompt perturbation, and candidate reordering?
RQ3: Can a fallback mechanism mitigate LLM failure while preserving quality gains?
RQ4: Does LLM reranking affect demographic fairness, and do diversity-oriented prompts help or harm?

2 Background and Related Work

Sequential Recommendation. SASRec [12] applies causal self-attention over user interaction sequences to predict next items. It is trained once on MovieLens-1M [7] - the most popular benchmark in the referenced studies - and held fixed. The focus of this project is what happens when an LLM is inserted after it.

LLM Reranking. Hou et al. [9] established the foundational approach: provide an LLM with a user's history and a candidate list, and ask it to rerank. They identified *position bias* - the LLM's tendency to favour items near the top or bottom of the input list - and proposed bootstrapped aggregation as a mitigation. Our results directly contradict their position bias finding for a strong instruct-tuned model (Section 5.3).

Jiang et al. [11] provide a comprehensive evaluation framework, measuring hallucination rates, position bias and sensitivity to history length across multiple LLMs on MovieLens1M, while also establishing model selection as a dependability decision. Gao et al. [6] introduce chain-of-thought reranking with multi-aspect prompts, assuming the LLM always produces valid output - an assumption our results show is model-dependent and fragile. Carraro and Bridge [3] use LLMs for diversity reranking, replacing hallucinated item IDs with random candidates. This is item-level patching, not a fallback mechanism - the system never decides to reject the LLM's output entirely.

Fairness. A prior literature review examined bias in recommender systems [4]. Three findings ground this project: biases span interaction data, sociodemographic, and dynamic reinforcement categories; fairness interventions redistribute unfairness rather than eliminating it [14, 15]; and "recommendation quality" is stakeholder-relative [4]. This motivates the multi-metric evaluation in Section 5, which tracks accuracy, diversity, popularity bias, and demographic fairness simultaneously.

Failure Modes. A second literature review conducted for this project identified four LLM failure modes in recommender pipelines: hallucination, non-determinism [1, 16], latency instability [10], and gray failure [2]. Deldjoo [5] measures temporal stability of LLM recommendations across sessions, finding it varies by model. Critically, stability is treated as a descriptive property - never correlated with quality and never used as a decision signal. This is the closest existing work to our self-consistency analysis, and the gap we aimed to fill.

Wu et al. [19] propose progressive failover with smaller model variants for infrastructure-level failures (crashes, timeouts). Their system is blind to *semantic* degradation - an LLM that returns valid but unhelpful output is

never caught. Zhang et al. [21] (UniGRF) take the opposite architectural approach, unifying retrieval and ranking into a monolithic model, eliminating the fault isolation boundaries our design relies upon.

The Gap This Project Addresses. A structured review of six key [3, 5, 6, 9, 11, 21] papers against five criteria - (1) quantifies per-query non-determinism, (2) correlates instability with quality, (3) proposes query-level deferral, (4) reports stability metrics, (5) frames reliability as a system concern - confirmed that no existing work does all five. No paper uses self-consistency or any behavioural signal as a per-query gate for accepting or rejecting LLM reranking output in a recommender pipeline.

3 System Design and Methodology

3.1 Four system comparison

System	Description	Behaviour
A (Baseline)	SASRec produces ranked top-20	No LLM involvement
B (Naive LLM)	LLM reranking, padded with SASRec items if incomplete	Always serves LLM output
C (Hard fallback)	LLM output if fully valid; SASRec fallback on any error	Conservative - discards any error
D (Tiered fallback)	LLM output if valid or recoverable; SASRec fallback on fatal errors	Distinguishes recoverable errors

The key insight motivating Systems C and D is that not all LLM failures are equally severe. This leads to the two-tier failure taxonomy described below.

3.2 Two-Tier Failure Taxonomy

LLM output errors are classified into two severity tiers:

Recoverable errors produce 10 valid, in-range items after minor post-processing. For example, if the LLM returns 10 items but includes a duplicate, removing the duplicate and appending the next SASRec candidate yields a usable list. The LLM's ranking judgment is largely preserved.

Fatal errors produce no usable ranking signal: no parseable output, all items out of range, or fewer items than required with no way to recover intent. The only safe option is full reversion to SASRec.

System C treats both tiers identically (revert on any error). System D reverts only on fatal errors, preserving the LLM's ranking for recoverable cases. This distinction has measurable quality consequences (Section 5.1).

3.3 Models

Three LLMs were selected to span a capability and compliance range:

- **Qwen2-7B-Instruct**: A 7B-parameter instruct-tuned model. Strong instruction-following. Run at temperatures 0.0, 0.1, and 0.7.
- **Nemotron-Mini-4B-Instruct**: A 4B-parameter instruct-tuned model. Weaker instruction-following. Run at temperatures 0.1 and 0.7.
- **gpt-oss-20b**: A 20B-parameter *base* model (not instruct-tuned). Requires a proprietary "harmony format" incompatible with our standardised prompt pipeline. Run at temperature 0.1.

All models were run locally on university BlueBear HPC infrastructure with limited capability and storage space (inference only, no fine-tuning). The infrastructure limitations motivated the selection of smaller scale 2024 - early 2025 open-weight model versions. The inclusion of gpt-oss in the report is deliberate: it represents a realistic integration failure where a model that functions correctly in its native environment produces zero valid output when deployed through a generic API.

3.4 Evaluation Protocol

For each of 500 test users, SASRec generates a top-20 candidate list. The LLM receives the user's watch history and these 20 candidates, returning a top-10 reranking. Each configuration is evaluated with:

- **Accuracy:** HR@10, NDCG@10, MRR@10
- **Diversity:** ILD (intra-list distance), genre coverage
- **Popularity bias:** ARP, PopRatio@80, Gini, catalog coverage
- **Fairness:** Per-gender HR and NDCG, with gap measurement

Self-consistency is measured by querying each model K=3 times per user with identical input, computing Kendall's τ across outputs, and correlating mean τ with ΔNDCG via Spearman rank correlation.

3.5 Perturbation Framework

To understand what *actually* destabilises LLM reranking, five perturbation types were applied:

1. **Run variance:** Same input, different random seeds (K=3 runs)
2. **Temperature variation:** $t \in \{0.0, 0.1, 0.7\}$
3. **Candidate order reversal:** All 20 candidates presented in reverse SASRec order
4. **Candidate random shuffle:** All 20 candidates randomly reordered
5. **Prompt rephrase:** Semantically equivalent rewording of the instruction

Temperature variation and run variance are drawn from the main experimental runs; candidate reordering and prompt rephrase are dedicated perturbation experiments run separately. Each perturbation's impact is measured as *sensitivity* $= 1 - \tau$ between the perturbed and baseline output, alongside quality change (ΔNDCG). This produces a complete sensitivity hierarchy (Section 5.3).

3.6 Diversity Prompt Experiment

Three prompt variants test whether LLMs can be steered toward diverse recommendations:

- **Main prompt:** Balanced instruction mentioning diversity and fairness alongside relevance

- **Diversity v1 (soft):** "Diversity is MORE IMPORTANT than pure relevance”
- **Diversity v2 (aggressive):** Explicit metric names - "Maximise GENRE COVERAGE, maximise INTRA-LIST DIVERSITY etc.”

All three are run with Qwen at t=0.1 on the same 500 users, enabling controlled comparison. Full prompts can be seen in the Appendix.

3.7 Displacement Analysis

To test whether the degree of reranking predicts quality, four displacement metrics are computed per user: item overlap between SASRec's top-10 and the LLM's top-10, mean rank displacement, count of items promoted from positions 11–20, and Kendall's τ between the two orderings. Mann-Whitney U tests compare these metrics between users whose quality improved versus those who were harmed, testing whether any feature provides a usable decision boundary.

4 Implementation

4.1 Data Collection and Logging

The core collection script (`rerank-dep.py`) logs per-query JSONL records containing: the raw LLM text output, parsed item IDs (`llm_item_ids`), SASRec-backfilled item IDs (`filled_item_ids`), validity classification with specific error type, and wall-clock latency. Critically, raw LLM output and backfilled output are stored separately - this design choice, made early in the project, proved essential. It enabled every post-hoc analysis (fallback simulation, self-consistency, displacement, error taxonomy) without re-running inference, saving substantial GPU time.

The script supports command-line arguments for `--run_index`, `--prompt_version`, `--temperature`, and append-mode resume, allowing systematic exploration of the experimental space. All runs are reproducible given the same model checkpoint and random seed.

4.2 Validity Classification and Error Taxonomy

Each LLM response passes through a multi-stage parser:

1. **Format extraction:** Attempt to parse a numbered list of movie IDs from the raw text
2. **Range validation:** Check all IDs exist in the candidate set
3. **Deduplication:** Identify and remove duplicate entries
4. **Length check:** Verify exactly 10 items remain (or can be recovered)

The parser assigns one of six error codes: `valid`, `duplicates_removed`, `out_of_range_removed` (recoverable), `no_parseable_output`, `all_out_of_range`, or `incomplete` (fatal). This classification drives the tiered fallback logic.

4.3 Four-System Evaluation Pipeline

The evaluation script reconstructs all four systems from stored raw outputs: System B uses `filled_item_ids` directly; System C gates on `is_valid`; System D gates on error severity. This analytical reconstruction means fallback policies can be changed and re-evaluated without re-running LLM inference.

4.4 Self-Consistency Analysis

For each user across K=3 runs: compute pairwise Kendall τ, take the mean as the consistency score, and correlate with ΔNDCG (LLM minus SASRec) via Spearman rank correlation. Fallback threshold sweeps report system-level NDCG at each consistency cutoff to test whether any threshold improves aggregate quality.

4.5 Perturbation and Displacement Analysis

Each perturbation experiment produces, per user, a sensitivity score ($1 - \tau$ between baseline and perturbed output) and a quality change (ΔNDCG). These are aggregated into a sensitivity hierarchy and tested for correlation: does higher sensitivity predict larger quality changes? Mann-Whitney U tests compare displacement metrics between users whose quality improved versus those who were harmed, testing whether any displacement feature discriminates between good and bad reranking.

4.6 Tools and Infrastructure

- **SASRec:** modern PyTorch reimplementation of a popular state-of-the-art solution, trained on MovieLens-1M - the most popular benchmarking dataset in similar papers. SASRec's own Steam Game Reviews dataset was used on the project's earlier stages to build the pipeline.
- **LLM inference:** HuggingFace Transformers, run on university BlueBear HPC. Model families selected based on Artificial Analysis [22] ranking of instruction-tuned open-weight models; versions downgraded after Qwen3.5's weights exceeded the HPC storage space quota
- **Evaluation:** Custom Python scripts using NumPy, SciPy (Kendall τ, Spearman ρ, Mann-Whitney U, Wilcoxon signed-rank) - metrics taken from reranking / dependability studies or recommended by consulted UoB professors; implementations generated with GenAI assistance

5 Results and Analysis

5.1 Failure Rates and Fallback Behaviour (RQ3)

Table 1 summarises validity rates and error types across all configurations.

Configuration	Valid	Recoverable	Fatal	Hard FB Rate	Tiered FB Rate
Qwen t=0.0	500	0	0	0%	0%
Qwen t=0.1	500	0	0	0%	0%
Qwen t=0.7	479	21	0	4.2%	0%
Nemotron t=0.1	225	0	275	55%	55%
Nemotron t=0.7	314	0	186	37.2%	37.2%
gpt-oss t=0.1	0	0	500	100%	100%

Table 1: Failure taxonomy across configurations. Qwen t=0.7's 21 failures are all `duplicates_removed`(recoverable). All Nemotron and gpt-oss failures are `no_parseable_output` (fatal).

Three patterns emerge. First, **instruction-tuning determines reliability**, not model scale: the 7B instruct-tuned Qwen achieves 100% validity at low temperatures, the 4B instruct-tuned Nemotron achieves 45%, and the 20B base model gpt-oss, despite being twice as large as Qwen, achieves 0%. Second, **the tiered taxonomy matters only for Qwen t=0.7**, where 21 users produce outputs with duplicates that are perfectly usable after deduplication. Hard fallback unnecessarily reverts these 21 users. Third, **Nemotron's failures are entirely fatal** - there is no intermediate case to recover, so C=D for Nemotron.

A surprising sub-finding: Nemotron is *more* valid at higher temperature (62.8% at t=0.7 versus 45% at t=0.1). Higher temperature may help a weak instruction-follower stumble into valid output format by exploring a wider distribution. Furthermore, Nemotron's failures are non-deterministic - across two runs at t=0.7, 117 users succeeded only in run 0, 108 only in run 1, and 78 in neither. The same user may fail or succeed depending on the random seed.

5.2 Reranking Quality (RQ1, RQ3)

Table 2 presents the four-way comparison

Configuration	System A	System B	Δ vs A	System C	System D
Qwen t=0.0	0.0655	0.0711	+8.5%	0.0711	0.0711
Qwen t=0.1	0.0655	0.0705	+7.6%	0.0705	0.0705
Qwen t=0.7	0.0655	0.0690	+5.3%	**0.0672**	0.0690
Nemotron t=0.1	0.0655	0.0630	-3.8%	0.0630	0.0630
Nemotron t=0.7	0.0655	0.0652	-0.5%	0.0652	0.0652
gpt-oss t=0.1	0.0655	0.0655	0.0%	0.0655	0.0655

Table 2: NDCG@10 across all four systems. Bold highlights where System C loses quality that System D preserves.

The critical result is the **Qwen t=0.7 row**: hard fallback (System C) drops NDCG from 0.0690 to 0.0672 - losing 0.0018 - by reverting 21 users whose output was perfectly serviceable after deduplication. Tiered fallback (System D) recovers this entirely, matching System B at 0.0690. The same pattern appears in MRR (0.0434 → 0.0412 under hard fallback, recovered to 0.0434 under tiered) and in per-gender NDCG (male NDCG drops from 0.0645 to 0.0620 under hard fallback, recovered under tiered). This demonstrates that **error severity classification has measurable quality and fairness consequences**.

For configurations where B=C=D (Qwen at low temperature, Nemotron, gpt-oss), the fallback mechanisms are transparent - they impose zero cost when not needed and activate correctly when they are. The gpt-oss row confirms that both fallback systems correctly handle total integration failure: every metric equals the SASRec baseline because every query falls back.

Qwen improves NDCG by 5–8.5% depending on temperature. Nemotron slightly *hurts* performance. HR@10 is nearly identical across all systems because the LLM can only reorder items already in the candidate set - it cannot introduce new relevant items.

5.3 Perturbation Sensitivity Hierarchy (RQ2)

Table 3 ranks all perturbation types by their destabilising effect, measured as mean sensitivity ($1 - \tau$).

Perturbation	Sensitivity	Item Overlap	ΔNDCG	Key Observation
Run variance, Qwen t=0.1	0.004	9.99/10	+0.001	Essentially deterministic
Diversity v1 vs main	0.004	9.94/10	+0.001	Soft instructions ignored
Candidate reversal, t=0.0	0.007	9.98/10	+0.002	Negligible position bias
Candidate reversal, t=0.1	0.007	9.98/10	+0.003	Negligible position bias
Candidate random shuffle	0.020	9.95/10	-0.001	Negligible position bias
Run variance, Qwen t=0.7	0.156	9.50/10	-0.002	Moderate stochasticity
Run variance, Nemotron t=0.7	0.488	8.24/10	-0.008	High instability
Diversity v2 vs main	0.726	7.81/10	-0.004	Aggressive prompt: major displacement
Prompt rephrase, t=0.1	**0.752**	**9.64/10**	**-0.002**	**Prompt wording dominates all others**

Table 3: Complete sensitivity hierarchy ordered by increasing destabilisation.

The headline finding: prompt wording is approximately 100× more destabilising than candidate input ordering (sensitivity 0.752 vs 0.007). This directly contradicts the emphasis on position bias in Hou et al. [9], who proposed bootstrapped candidate ordering as a primary mitigation. For a strong instruct-tuned model, candidate ordering is essentially irrelevant - reversing all 20 candidates at t=0.0 produces identical output for 495 of 500 users (99%).

Moreover, the prompt rephrase - a semantically equivalent rewording - caused 92 of 500 users (18.4%) to shift from valid to invalid output, with zero users moving in the opposite direction. Among the 408 users where both prompts produced valid output, quality was statistically identical (Wilcoxon p=0.97, overlap 9.64/10). This demonstrates that **pipeline reliability depends on exact prompt wording, not just semantic content**, a fragility with implications for production deployment.

Sensitivity significantly correlates with the *magnitude* of quality change (|ΔNDCG|) across multiple perturbation types (all $p < 0.005$), but never with quality *direction* (ΔNDCG). It functions as a risk indicator - flagging users likely to experience large quality swings - but cannot distinguish improvement from degradation.

5.4 Self-Consistency and Displacement: Two Negative Results (RQ3)

Self-consistency. Table 4 shows the core negative result.

Configuration	Spearman ρ (τ vs ΔNDCG)	p-value	Significant?
Qwen t=0.0	NaN (τ=1.0 for all)	-	No variance
Qwen t=0.1	-0.087	0.053	No

Qwen t=0.7	-0.031	0.497	No
Nemotron t=0.1	-0.050	0.474	No
Nemotron t=0.7	-0.006	0.923	No

Table 4: Self-consistency (mean pairwise Kendall τ across K=3 runs) shows no significant correlation with reranking quality at any configuration.

The hypothesis fails for two distinct reasons. For Qwen: a **ceiling effect** - the model is near-perfectly deterministic at low temperatures, leaving no variance for a gate to exploit. At t=0.0, τ=1.0 for every user. This contrasts with Ouyang et al. [16], who found non-determinism persists at t=0 for ChatGPT in code generation. The discrepancy likely reflects differences in deployment (local single-GPU inference vs cloud API) and task structure (constrained ranking over 20 items vs open-ended code generation). For structured reranking with locally-hosted models, temperature zero achieves perfect determinism. For Nemotron: non-determinism is **temperature-driven, not query-driven** - different users fail on different runs with no predictable pattern. Additionally, 86% of users have ΔNDCG=0 due to the leave-one-out evaluation protocol (only one test item per user), creating massive ties that suppress any correlation.

Displacement. Rank displacement - how much the LLM changes SASRec's ordering - was investigated as an alternative signal. Some weak Spearman correlations between displacement and quality reach significance in individual configurations (e.g., Qwen t=0.1 overlap ρ=+0.128, p=0.004), but these are inconsistent across models and temperatures. More importantly, all Mann-Whitney U tests comparing displacement between improved and harmed users are non-significant (all $p > 0.29$), meaning no displacement feature provides a usable decision boundary.

Interestingly, conservative reranking outperforms aggressive reranking: Qwen (overlap 9.99/10) improves NDCG by 8.5%, while Nemotron (overlap 8.7/10, swapping more items from positions 11–20) performs worse than the SASRec baseline.

5.5 Diversity Prompting and Fairness (RQ4)

Table 5 compares the three prompt variants on diversity, popularity, and fairness metrics

Metric	**SASRec**	**Main**	**Div v1 (Soft)**	**Div v2 (Aggressive)**
NDCG@10	0.0655	0.0705	0.0711	0.0662
ILD	0.6510	0.6509	0.6521	0.6482 <-
GenreCov	7.642	7.638	7.650	7.614 <-
ARP	1037.3	1037.7	1038.6	998.5 <<-
Coverage	0.3216	0.3216	0.3233	0.3368 ->
NDCG Gap (M–F)	0.0220	0.0197	0.0188	0.0034
NDCG Female	0.0811	0.0844	0.0844	**0.0637 <<-**
HR Female	0.1849	0.1849	0.1849	**0.1438 <<-**

Table 5: Diversity prompt comparison (all Qwen t=0.1, 500 users). Arrows indicate direction relative to SASRec baseline.

Three findings emerge:

1. **Soft diversity instructions are ignored.** Div v1 is statistically indistinguishable from the main prompt on every metric. The perturbation analysis confirms this: sensitivity between them is 0.004 with 9.94/10 item overlap. The LLM does not change its behaviour in response to "diversity is more important than relevance."
2. **The LLM conflates diversity with obscurity.** Div v2 successfully reduced popularity bias (ARP dropped 3.7%, catalog coverage rose 4.7%), but the two metrics it was explicitly instructed to maximise - ILD and genre coverage - actually decreased. The model interpreted "diversity" as "recommend lesser-known items" rather than "spread across genres." It debiased popularity while making genre diversity marginally worse.
3. **Apparent fairness improvement masks degradation.** Div v2's dramatic narrowing of the NDCG gender gap (0.0220 → 0.0034) looks like a fairness success. Decomposed: male NDCG improved slightly (0.0591 → 0.0672) while female NDCG collapsed (0.0811 → 0.0637), and female HR@10 dropped from 0.1849 to 0.1438. The gap narrowed because the previously-advantaged group was harmed, not because the disadvantaged group benefited. This is a known fairness anti-pattern -fairness through degradation - that aggregate gap metrics conceal.

It is worth noting that the LLM never sees user demographics. All fairness effects are incidental to the reranking process, making the standard prompt's gap reduction (0.0220 → 0.0197) a genuinely interesting result: Qwen improves quality for both groups while slightly narrowing the gap, without being instructed to do so.

6 Discussion

6.1 The Hypothesis That Failed - and Why That Matters

The original plan was to build a self-consistency gate: query the LLM multiple times, measure agreement, and reject inconsistent output. This would have been an elegant, model-agnostic reliability mechanism. It failed because LLM non-determinism in structured ranking tasks is qualitatively different from non-determinism in open-ended reasoning. In reasoning tasks, uncertainty produces diverse answers; in ranking tasks with strong instruct-tuned models, temperature either produces identical output (low t) or random perturbation uncorrelated with quality (high t). The ranking task's output space is too constrained for self-consistency to be informative. Our next hypothesis was to implement displacement check in the fallback validation to save tokens when a model does not engage in reranking actively. This, however, was also disproven by the conservative Qwen outperforming more diverse Nemotron.

These negative results turned this project from a novel algorithm into an empirical study. Self-consistency as a reliability signal is an obvious idea that others will attempt. Documenting why it fails - and under what structural conditions (leave-one-out ties, ceiling effects, temperature-driven rather than query-driven variance) will future researchers avoid this dead end.

6.2 Error Severity Classification

The most practically useful finding is the tiered failure taxonomy. It requires no repeated LLM calls, no statistical analysis, and no threshold tuning - just a parser that distinguishes "this output has a fixable problem" from "this output is unusable." For Qwen t=0.7, this distinction recovers 0.0018 NDCG and 0.0022 MRR that hard fallback unnecessarily discards. The mechanism is cheap, deterministic, and zero-cost when not triggered.

6.3 Implications for Prompt Engineering

The perturbation hierarchy reveals that the dominant source of instability in LLM reranking is not what the literature suggests. Position bias - the focus of Hou et al. [9] - is negligible for strong instruct-tuned models. Prompt wording is the critical variable: a semantically equivalent rephrase causes 100× more output change than reversing the entire candidate list, and breaks 18.4% of outputs. This suggests that prompt engineering for LLM reranking should be treated as a reliability engineering problem, not just a quality optimisation problem.

6.4 Limitations

Evaluation method. Leave-one-out evaluation means 86% of users are ΔNDCG=0 ties. This structurally limits the discriminative power of any per-query signal. A multi-item holdout protocol would provide finer-grained quality measurement.

Dataset scope. All experiments use MovieLens-1M. Generalisation to other domains (e-commerce, news) is not established.

Missing baselines. A non-LLM deterministic diversity reranker would have clarified whether diversity invariance is a language model limitation or a candidate-pool limitation. This was not implemented due to time constraints.

Statistical power. With 500 test users and small effect sizes, some findings may lack power to detect weak effects. The strong findings (position bias negligibility, prompt rephrase fragility, self-consistency failure) are robust; subtler effects would benefit from larger samples.

7 Conclusions and Future Work

7.1 Answers to Research Questions

RQ1: Model-dependent. Qwen improves NDCG@10 by 5-8% through conservative reordering; Nemotron degrades performance slightly through more aggressive displacement. A base model (gpt-oss) produces zero valid output, representing an integration failure.
RQ2: Qwen is near-perfectly deterministic at t <=0.1; with prompt wording as the dominant instability source (100× larger effect than candidate ordering). Position bias is negligible for strong instruct-tuned models, contradicting prior findings. Nemotron exhibits high, non-deterministic instability.
RQ3: Hard validity checking is highly effective for Nemotron. Self-consistency and displacement are not a viable reliability signal. The tiered failure taxonomy - distinguishing recoverable from fatal errors - recovers quality that hard fallback unnecessarily discards.
RQ4: Qwen incidentally reduces the gender fairness gap without seeing demographics. Aggressive diversity prompting creates a *fairness through degradation* pattern: the gap narrows because female users are harmed, not because male users improve. Soft diversity instructions are ignored entirely.

7.2 Future Work

The stability-based fallback was not implemented as a deployable component. The empirical findings explain why: self-consistency, the originally planned gate signal, proved non-viable.

While it was highlighted to me by the professors consulted (Matthew Leeke and Phillip Smith) that creating a novel fallback mechanism with a brand new anomaly detection solution might be too complex for a final year project, my supervisor has encouraged me to continue working on the topic after the submission deadline and hopefully submit a short paper for a SIGIR workshop. More experiments and consequent analysis should be

conducted for that. Even though a complex algorithm has not been created, a strong empirical study can serve as a foundation for one.

7.3 Reflection

Originally aimed at novel algorithm development, this project changed its format into an empirical study after the main hypotheses - using self-consistency and low displacement for anomaly detection - have not been supported. The decision to log raw LLM outputs separately from post-processed outputs, made early for pragmatic reasons, enabled every subsequent analysis without re-running expensive GPU inference. The negative results required confirmation, evaluation and analysis - transforming my work from a failed project into an empirical study. The most practically useful finding - the tiered failure taxonomy - was not in the original project plan at all; it emerged from carefully categorising the errors the LLM actually produced.

As for the final year project's goals, I also believe this change of direction indicates adaptability in a professional setting.

References

1. Astekin, M., Hort, M., & Moonen, L. (2024). An Exploratory Study on How Non-Determinism in Large Language Models Affects Log Parsing. *Proceedings - 2024 IEEE/ACM 2nd International Workshop on Interpretability, Robustness, and Benchmarking in Neural Software Engineering, InteNSE 2024*, 13–18. https://doi.org/10.1145/3643661.3643952
2. Cai, Q., Cao, J., Xu, G., & Zhu, N. (2024). Distributed Recommendation Systems: Survey and Research Directions. *ACM Transactions on Information Systems*, *43*(1). https://doi.org/10.1145/3694783
3. Carraro, D., & Bridge, D. (2026). Enhancing Recommendation Diversity by Re-ranking with Large Language Models. *ACM Transactions on Recommender Systems*, *4*(2), 1–40. https://doi.org/10.1145/3700604
4. Chen, J., Dong, H., Wang, X., Feng, F., Wang, M., & He, X. (2023). Bias and Debias in Recommender System: A Survey and Future Directions. *ACM Transactions on Information Systems*, *41*(3). https://doi.org/10.1145/3564284
5. Deldjoo, Y. (2024). *Understanding Biases in ChatGPT-based Recommender Systems: Provider Fairness, Temporal Stability, and Recency* (Vol. 1). https://doi.org/XXXXXXX.XXXXXXX
6. Gao, J., Chen, B., Liu, W., Li, X., Wang, Y., Wang, W., Guo, H., Tang, R., & Zhao, X. (n.d.). LLM4Rerank: LLM-based Auto-Reranking Framework for Recommendations CCS Concepts • Information systems → Retrieval models and ranking; Data mining. Keywords Reranking, Recommender System, Large Language Model. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation emai (WWW '25)* (Vol. 1). https://doi.org/XXXXXXX.XXXXXXX
7. Harper, F.M. & Konstan, J.A. (2015). The MovieLens Datasets. *ACM TIIS*, 5(4), 1–19.
8. Hou, Y., Kou, Y., Shen, D., Zhou, X., Li, D., Nie, T., & Yu, G. (2025). LeadFairRec: LLM-enhanced Discriminative Counterfactual Debiasing for Two-sided Fairness in Recommendation. *CIKM 2025 - Proceedings of the 34th ACM International Conference on Information and Knowledge Management*, 867–876. https://doi.org/10.1145/3746252.3761126
9. Hou, Y., Zhang, J., Lin, Z., Lu, H., Xie, R., McAuley, J., & Zhao, W. X. (2024). *Large Language Models are Zero-Shot Rankers for Recommender Systems*. http://arxiv.org/abs/2305.08845
10. Jain, K., Parayil, A., Mallick, A., Choukse, E., Qin, X., Zhang, J., Goiri, Í., Wang, R., Bansal, C., Rühle, V., Kulkarni, A., Kofsky, S., & Rajmohan, S. (2025). Performance Aware LLM Load Balancer for Mixed Workloads. *EuroMLSys 2025 - Proceedings of the 2025 5th Workshop on Machine Learning and Systems*, 19–30. https://doi.org/10.1145/3721146.3721947
11. Jiang, C., Wang, J., Ma, W., Clarke, C. L. A., Wang, S., Wu, C., & Zhang, M. (2025). Beyond Utility: Evaluating LLM as Recommender. *WWW 2025 - Proceedings of the ACM Web Conference*, 3850–3862. https://doi.org/10.1145/3696410.3714759
12. Kang, W.-C., & McAuley, J. (2018). *Self-Attentive Sequential Recommendation*. http://arxiv.org/abs/1808.09781
13. Kieu, H. D., Nguyen, M. D., Nguyen, T. S., & Le, D. D. (2025). Keyword-driven Retrieval-Augmented Large Language Models for Cold-start User Recommendations. *WWW Companion 2025 - Companion Proceedings of the ACM Web Conference 2025*, 2717–2721. https://doi.org/10.1145/3701716.3717855
14. Krause, T., Deriyeva, A., Beinke, J. H., Bartels, G. Y., & Thomas, O. (2025). Mitigating Exposure Bias in Recommender Systems—A Comparative Analysis of Discrete Choice Models. *ACM Transactions on Recommender Systems*, *3*(2), 1–37. https://doi.org/10.1145/3641291
15. Malitesta, D., Medda, G., Purificato, E., Marras, M., Malliaros, F., & Boratto, L. (2025). How Fair is Your Diffusion Recommender Model? *RecSys2025 - Proceedings of the 19th ACM Conference on Recommender Systems*, 1199–1205. https://doi.org/10.1145/3705328.3759318
16. Ouyang, S., Zhang, J. M., Harman, M., & Wang, M. (2025). An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Transactions on Software Engineering and Methodology*, *34*(2). https://doi.org/10.1145/3697010
17. Saberian, M., & Basilico, J. (2021). RecSysOps: Best practices for operating a large-scale recommender system. *RecSys 2021 - 15th ACM Conference on Recommender Systems*, 590–591. https://doi.org/10.1145/3460231.3474620

18. Soldani, J., & Brogi, A. (2022). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Computing Surveys*, *55*(3). https://doi.org/10.1145/3501297
19. Wu, L., Hanafy, W., Abdelzaher, T., Irwin, D., Milzman, J., & Shenoy, P. (2025). *FailLite: Failure-Resilient Model Serving for Resource-Constrained Edge Environments*. 416–429. https://doi.org/10.1145/3772052.3772243
20. Xiong, Y., Jiang, Y., Yang, Z., Qu, L., Zhao, G., Liu, S., Zhong, D., Pinzur, B., Zhang, J., Wang, Y., Jose, J., Pourreza, H., Baxter, J., Datta, K., Ram, P., Melton, L., Chau, J., Cheng, P., Xiong, Y., & Zhou, L. (2025). SuperBench: A Proactive Validation System for Improving Reliability of Cloud AI Infrastructure. *ACM Transactions on Computer Systems*. https://doi.org/10.1145/3767334
21. Zhang, L., Song, K., Lee, Y. Q., Guo, W., Wang, H., Li, Y., Guo, H., Liu, Y., Lian, D., & Chen, E. (2025). Killing Two Birds with One Stone: Unifying Retrieval and Ranking with a Single Generative Recommendation Model. *SIGIR 2025 - Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2224–2234. https://doi.org/10.1145/3726302.3730017
22. Artificial Analysis, https://artificialanalysis.ai/

Appendix

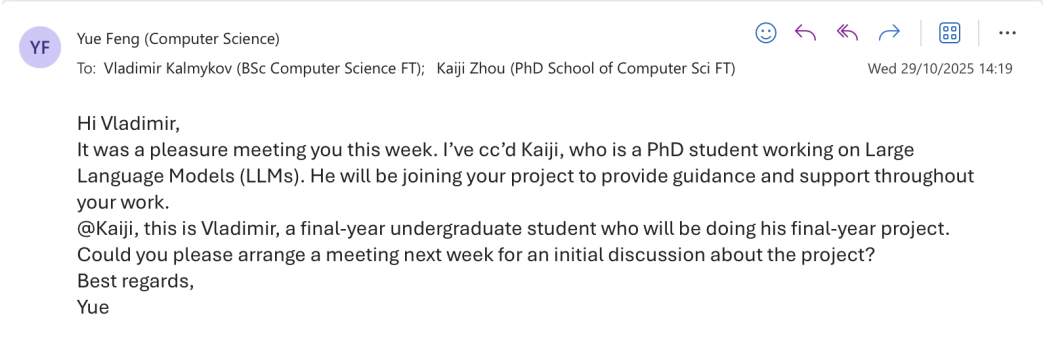
Project progress timeline

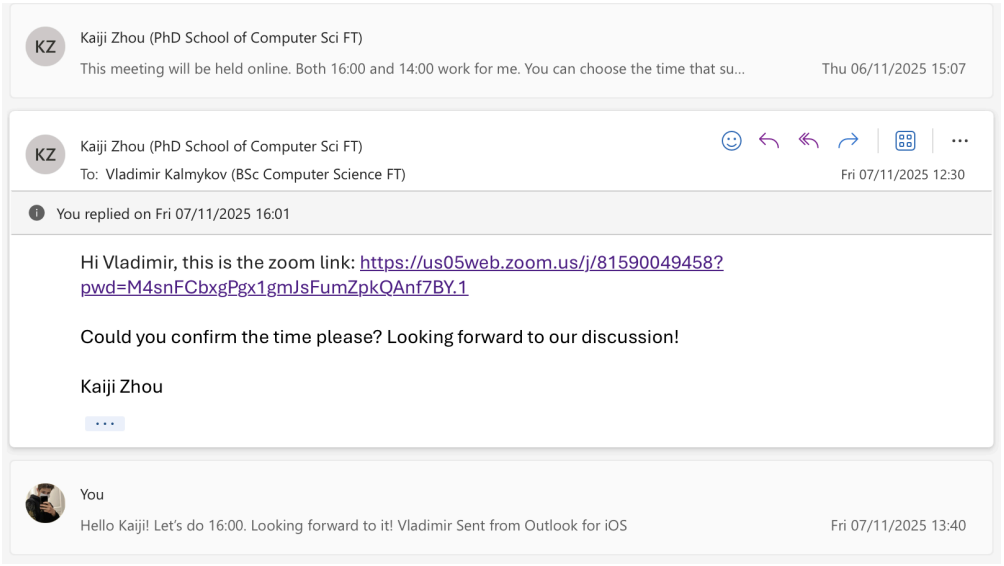
Since my commits to gitlab were not very regular and a very large proportion of the work done was reading research, I will be providing evidence of work with commentary here.

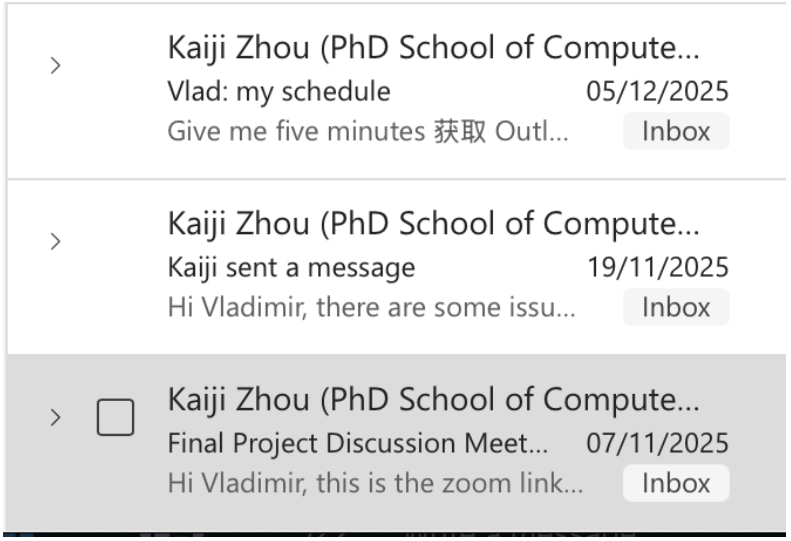
The majority of the files has only been uploaded to the git repository since March 9 due to both slight technical issues and my unawareness of regular commits' importance for the marking. However, both theoretical and practical work on the project has been conducted before then with evidence documented using other means.

A brief timeline of repo's contents is provided below

- **The semester 1:** I only managed to get in contact with my supervisor at the end of October. We had a meeting where we discussed my general areas of interest, after which I was assigned a PhD student Kaiji Zhou, a brilliant person who was essentially responsible for my project supervision. We agreed to have meetings every 2 weeks and went on to meet 3 times throughout weeks 6-11 of semester 1. Since the supervisor suggested developing a novel work, this time was almost entirely dedicated to researching the recent LLM and GenAI research and brainstorming ideas. Images confirming our regular meetings in the described period:



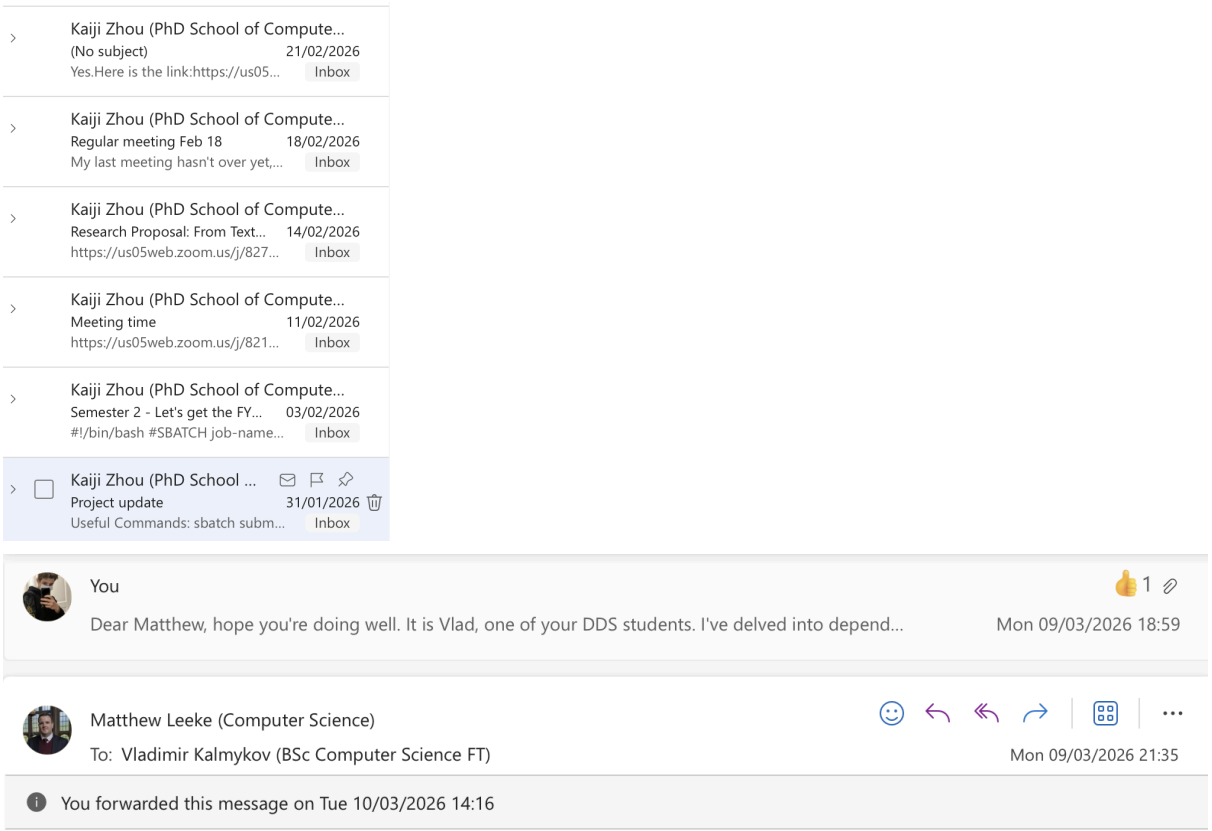




- **Semester 2** Kaiji has agreed to meet me twice a week and we went on to have 19 meetings over the course of the semester. I took notes during each one and can display them upon request; regular communication can also be demonstrated as above, if necessary.
- **January** of semester 2 was dedicated to narrowing the topic and doing reading, still looking for potentially novel work during **Weeks 1-2**. The result of this is the research proposal which was uploaded on gitlab on February 6 as the original readme (currently /research-pdfs/Original-Proposal.pdf). By that time we've decided to focus on LLM in the recommender setting. During **Week 3** I received access to the University of Bristol's HPC Isambard, where I started assembling the experiment pipeline and experimenting with SASRec and HuggingFace transformers. I used SASRec's own dataset, not Movielens, during that. The *.out files in the /logs or tidier /key-logs folder have timestamps confirming said activity, ranging approximately from **Week 3 to Week 5**
- After assembling the pipeline, I needed to find a specific practical task for the LLM to frame my project around. I spent **Week 6** doing reading on Bias and Fairness in Recommender systems and producing a literature review (/research-pdfs/Bias-Research.pdf) on the topic, reviewed by Mubashir Ali
- During **Week 7** and partly **Week 8** we were disconnected from the University of Bristol's supercomputer and had to transition to UoB's BlueBear. I had no access during the time, so I took the time consulting different professors (Mubashir Ali and Matthew Leeke) on the experiments' direction and narrowing the necessary evaluation metrics. I also drafted a literature review on recommender dependability (/research-pdfs/Recommender-Dependability-Research.pdf), completing the project's

theoretical foundation and establishing dependability experiments like stability under perturbations as the main focus of my experiments. Together with Kaiji we wrote /research-pdfs/Final-Project-Specification.pdf

- Starting with **Week 8,** when I got access to UoB's BlueBear HPC and rebuilt my experimental pipeline there, I found out about the importance of regular committing for the evaluation, and hence started doing so. Still, just in case - I spend the following time running all my experiments and drafting this report.
- I've kept exchanging info related to my project with many people during **Semester 2** - so I'll attach few confirmations of communication. The inspection may question me on that additionally, if necessary, and I will provide more.



Prompt variations

Main:

```
prompt = f"""You are a movie recommendation system. Your goal is to rerank
candidate movies to best match the user's preferences while ensuring diversity and
fairness.

The user has recently watched:
```

```
{history_str}

Here are candidate movies to rerank. Select and rerank the top 10 that best match
this user's taste while also including diverse genres and avoiding
over-recommending only popular movies:

{candidates_str}

Your primary task is to minimize the demographic, exposure and popularity biases.

Return ONLY the numbers (1-{num}) in your recommended order as a JSON list, like:

[3, 7, 1, 9, 5, 2, 8, 4, 10, 6]

Your top 10 reranked list:"""
```

Rephrase:

```
prompt = f"""As a film recommendation engine, reorder the following candidates to
suit this viewer's preferences. Prioritise variety and reduce bias toward popular
titles.

Viewing history:
{history_str}

Candidates to reorder (pick your best 10):
{candidates_str}

Output the candidate numbers (1-{num}) as a JSON list in your preferred order.

Reordered list:"""
```

Soft diversity:

```
prompt = f"""You are a movie recommendation system. Your goal is to rerank
candidate movies to maximise DIVERSITY in the final list.

The user has recently watched:
{history_str}

Here are candidate movies to rerank. Select the top 10 that:
1. SPREAD ACROSS AS MANY DIFFERENT GENRES as possible (maximise genre coverage)
2. MIX popular and niche/obscure titles (avoid all-blockbuster lists)
3. Still reasonably match the user's taste
```

```
Diversity is MORE IMPORTANT than pure relevance. A good list covers 5+ distinct
genres.

Candidates:
{candidates_str}

Return ONLY the numbers (1-{num}) in your recommended order as a JSON list, like:
[3, 7, 1, 9, 5, 2, 8, 4, 10, 6]

Your top 10 diversity-optimised list:"""
```

Metric-specific diversity:

```
prompt = f"""You are a diversity-focused movie recommender. Your PRIMARY objective
is to maximise the variety of genres in your recommendations. Your SECONDARY
objective is relevance to the user.

The user has recently watched:
{history_str}

Candidates:
{candidates_str}

INSTRUCTIONS:
- Select exactly 10 movies
- Maximise GENRE COVERAGE: include as many different genres as possible (aim for 8+
unique genres across your 10 picks)
- Maximise INTRA-LIST DIVERSITY: consecutive movies should NOT share genres. Each
pair of movies in your list should be as genre-dissimilar as possible
- Include at least one movie from a genre the user has NOT watched before
- Aim to improve the Gini index score
- Prefer lesser-known titles over blockbusters when relevance is similar

Return ONLY the numbers (1-{num}) as a JSON list.

Your diversity-maximised list:"""
```